UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3069 - Computación Paralela y Distribuida



Proyecto 02

Búsqueda de claves AES-128 con Open MPI:
distribución estática y estrategia por familias

Daniel Estrada - 20853

Andre Pivaral - 23574

Sebastian Estrada - 21405

Guatemala, 27 de octubre de 2026

Ciclo 2 de 2026

Índice

1. Introducción y objetivos	3
2. Antecedentes conceptuales	4
3. Diseño e implementación	5
4. Inciso 1: cifrado y descifrado	6
5. Estrategia base: lotes estáticos	7
6. Alternativa: familias de claves	8
7. Comunicación y pseudocódigo	9
8. Metodología experimental	10
9. Resultados medidos	11
10. Estimaciones y speedup	12
10.1. Control de distribución	13
11. Discusión de resultados	14
12. Conclusiones y recomendaciones	15
13. Referencias bibliográficas	16
Anexo 1. Funciones AES y archivos	17
Anexo 1. Funciones de búsqueda y MPI	18
Anexo 1. Familias y herramientas	19
Anexo 1. Librerías y comunicación	20
Anexo 2. Compilación	21
Anexo 2. Cifrado y descifrado	22
Anexo 2. Verificación	23
Anexo 2. Captura del caso a)	24
Anexo 2. Captura del caso b)	25
Anexo 2. Captura del caso c)	26
Anexo 2. Mediciones individuales	27
Apéndice. Reproducción y entrega	28

Criterio de lectura

Las mediciones proceden de las evidencias conservadas en el repositorio. Los valores proyectados se identifican como estimaciones; no representan búsquedas completadas. Las limitaciones y hallazgos de validación se exponen en la discusión y en los anexos.

1. Introducción y objetivos

La búsqueda de una clave por fuerza bruta constituye un problema adecuado para estudiar computación paralela: cada candidata puede probarse de forma independiente, pero el orden del recorrido y la comunicación necesaria para detener el trabajo afectan el tiempo de respuesta. Este proyecto adapta los programas del Laboratorio 03 para cifrar archivos de texto con AES-128 y comparar dos estrategias de búsqueda mediante cuatro procesos MPI.

La estrategia base distribuye lotes consecutivos y sincroniza los resultados al final de cada ronda. La alternativa combina familias de candidatas con asignación dinámica y notificación no bloqueante. El estudio utiliza el mensaje y las tres claves establecidos en el enunciado [7].

Objetivo general

Implementar y evaluar estrategias de distribución del trabajo con Open MPI para reducir el tiempo de búsqueda de claves numéricas utilizadas en un cifrado AES-128.

Objetivos específicos

1. Cargar un archivo .txt, cifrarlo con una clave de entrada y recuperar sus bytes originales con la misma clave.

2. Implementar una referencia secuencial y una búsqueda paralela base que reconozcan la frase «es una prueba de».

3. Comparar las claves 123456, 18014398509481984 y 18014398509481985 con cuatro procesos en las ejecuciones MPI.

4. Explicar el reparto, la búsqueda y la comunicación de una estrategia alternativa mediante diagramas y pseudocódigo.

5. Analizar tiempos, speedups, costos de coordinación y límites de las mediciones, con evidencia verificable.

Alcance

El cifrado usa claves AES de 16 bytes construidas desde enteros de hasta 64 bits. Se estudia un subconjunto de claves, no todo el espacio de 2^128 posibilidades. Los mensajes admitidos miden hasta 4096 bytes. Las ejecuciones registradas se realizaron en WSL; no constituyen una evaluación de escalabilidad en un clúster de varias máquinas.

2. Antecedentes conceptuales

AES-128 y representación de la clave

AES es un cifrador simétrico por bloques. FIPS 197 define bloques de 128 bits y variantes con claves de 128, 192 y 256 bits [1]. En este proyecto, **make_key** inicializa 16 bytes en cero y coloca el entero candidato en los últimos ocho, en orden big endian. Por ejemplo, 123456 se representa como `0000000000000000000000000001e240`. El arreglo cumple la longitud de AES-128, aunque la selección de claves tiene menos entropía. NIST indica: «Each of them transforms data in blocks of 128 bits».[1]

Modo ECB y relleno

El modo ECB cifra cada bloque por separado [2]. La implementación usa la interfaz EVP de OpenSSL y relleno PKCS#7: se añaden entre 1 y 16 bytes para completar el bloque, incluso cuando la longitud original ya es múltiplo de 16 [3]. El formato AESECB2 añade HMAC-SHA256 sobre versión, longitud y cifrado, con clave MAC derivada mediante un dominio distinto [8].

Fuerza bruta y validación por frase

Una búsqueda exhaustiva enumera candidatas, descifra el mensaje y aplica un criterio de aceptación. Aquí se busca la secuencia «es una prueba de» en el resultado, como exige el enunciado. Durante la búsqueda se desactiva la comprobación de relleno y se evalúa la frase sobre el texto original declarado. Solo si aparece se verifica el HMAC; así se conserva el criterio del enunciado sin aceptar falsos positivos de frases cortas.

MPI y coordinación del trabajo

Cada proceso conserva su propio contexto AES y su copia del cifrado. La base usa una reducción colectiva para compartir el hallazgo: **MPI_Allreduce** combina valores y entrega el resultado a todos [4]. La alternativa obtiene bloques mediante una operación atómica sobre una ventana remota [5], y utiliza envíos no bloqueantes para avisar un acierto [6].

Tiempo, speedup y eficiencia

```
R = claves probadas / tiempo de búsqueda
S_base = T_secuencial / T_base
E_base = S_base / 4
G_alt/base = T_base / T_alternativa
```

S compara la referencia secuencial con la base. G compara estrategias con distinto orden de candidatas; debe interpretarse como una razón de tiempos. Cuando interviene una proyección, la razón también es estimada.

1. NIST (2023), FIPS 197-upd1, resumen. Cita en inglés; enlace completo en la referencia [1].

3. Diseño e implementación

Organización del proyecto

Componente	Responsabilidad
laboratorio03/	Conserva los programas originales AES-128-GCM usados como punto de partida.
proyecto02/src/	aes_archivo; búsqueda secuencial; base MPI; alternativa MPI.
proyecto02/include/	AES y formato de archivo; argumentos y frase; preparación común de MPI.
proyecto02/tests/	Pruebas de archivos, búsquedas y generación de la bitácora de tiempos.
evidencias/	Capturas originales, comandos, salidas, archivos cifrados y tablas de resultados.

Entrada, salida y formato

El programa **aes_archivo** recibe operación, archivo de entrada, archivo de salida y clave decimal. La salida se abre en modo exclusivo para evitar sobrescribir archivos existentes. El contenedor de texto **AESECB2** almacena una cabecera, la longitud original y el cifrado hexadecimal. Esta representación permite mostrar el texto cifrado en las evidencias.

```
AESECB2
<longitud original en bytes>
<cifrado hexadecimal>
<etiqueta HMAC-SHA256 hexadecimal>
```

El cambio respecto al Laboratorio 03 es explícito: los archivos AESLAB1 del laboratorio (GCM), y los antiguos AESECB1 sin autenticación, no son compatibles con AESECB2. Se rechazan en lugar de interpretarlos con un esquema distinto.

Claves de los casos de estudio

Caso	Entero decimal	Clave AES de 16 bytes (hexadecimal)
a)	123456	00000000000000000000000001e240
b)	18014398509481984	00000000000000000040000000000000
c)	18014398509481985	00000000000000000040000000000001

Programación defensiva y memoria

Se verifican números decimales, longitudes, formato, archivos ausentes y retornos de EVP. Los arreglos de texto/cifrado son acotados y los contextos EVP se liberan al finalizar. El proceso 0 valida las entradas MPI y difunde el resultado a los demás para evitar que unos continúen y otros abandonen antes de una operación colectiva.

4. Inciso 1: cifrado y descifrado

El archivo se lee como bytes para conservar codificación y saltos de línea. En el cifrado se convierte la clave, se aplica AES-128-ECB con relleno y se guarda el contenedor. En el descifrado se carga el cifrado, se procesa con la clave recibida y se escribe el texto recuperado.

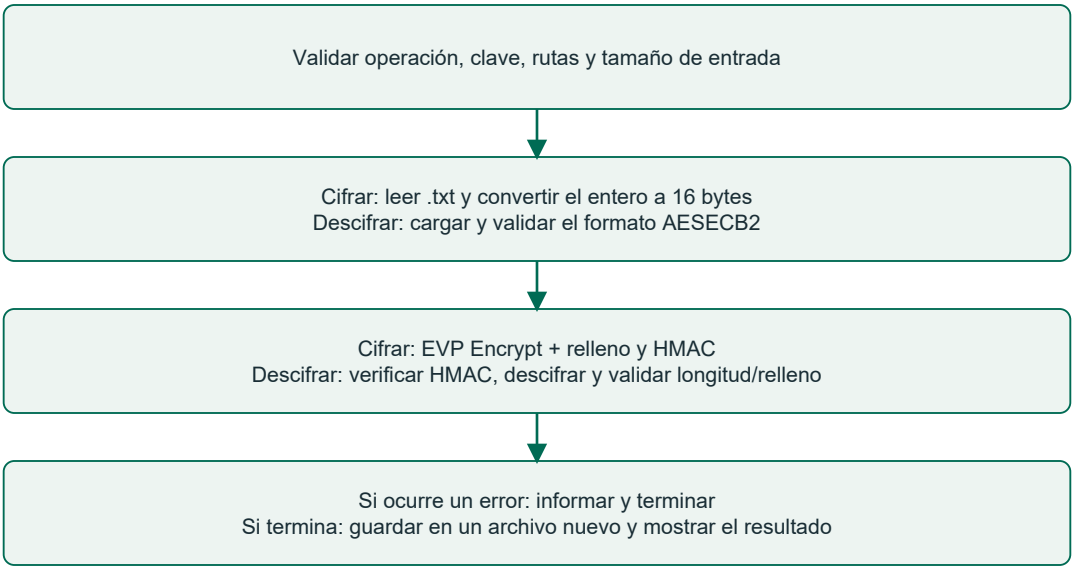


Figura 1. Flujo de procesamiento de archivos. Elaboración propia a partir de aes_archivo.c.

Verificación de recuperación

```
./proyecto02/build/aes_archivo cifrar entrada.txt salida.aes 123456
./proyecto02/build/aes_archivo descifrar salida.aes recuperado.txt 123456
cmp entrada.txt recuperado.txt
sha256sum entrada.txt recuperado.txt
```

Las capturas del Anexo 2 muestran texto original, cifrado, texto recuperado y hashes iguales. La suite del inciso 1 contiene 24 casos, entre ellos archivo vacío, UTF-8, saltos CRLF, 4096 bytes, exceso de tamaño, formatos dañados y protección de la entrada.

Autenticación y regresión corregida

La revisión inicial detectó que una clave incorrecta podía producir relleno válido. Se corrigió con HMAC-SHA256 en esquema encrypt-then-MAC: la etiqueta autentica el formato, la longitud y el cifrado antes de recuperar el archivo. La comparación usa CRYPTO_memcmp. El caso 44 se rechaza sin crear salida; también se verifican cambios en el primer bloque, en la etiqueta y en la longitud. Un vector independiente de clave cero y bloque cero comprueba AES-128-ECB, además de la recuperación [3, 8].

5. Estrategia base: lotes estáticos

Con n = 4 procesos y B = 65536 claves por lote, el proceso p recibe en la ronda r el intervalo [offset + (r·n+p)·B, offset + (r·n+p+1)·B), recortado al fin del rango. Los lotes de una ronda son disjuntos. Al terminar, una reducción MIN selecciona la clave encontrada; UINT64_MAX representa ausencia de acierto.

Ronda	P0	P1	P2	P3
0	[0, B)	[B, 2B)	[2B, 3B)	[3B, 4B)
1	[4B, 5B)	[5B, 6B)	[6B, 7B)	[7B, 8B)
r	Desde 4rB	Desde (4r+1)B	Desde (4r+2)B	Desde (4r+3)B

Tabla 1. Reparto base para offset = 0; B = 65536. No es una partición en cuatro bloques gigantes.

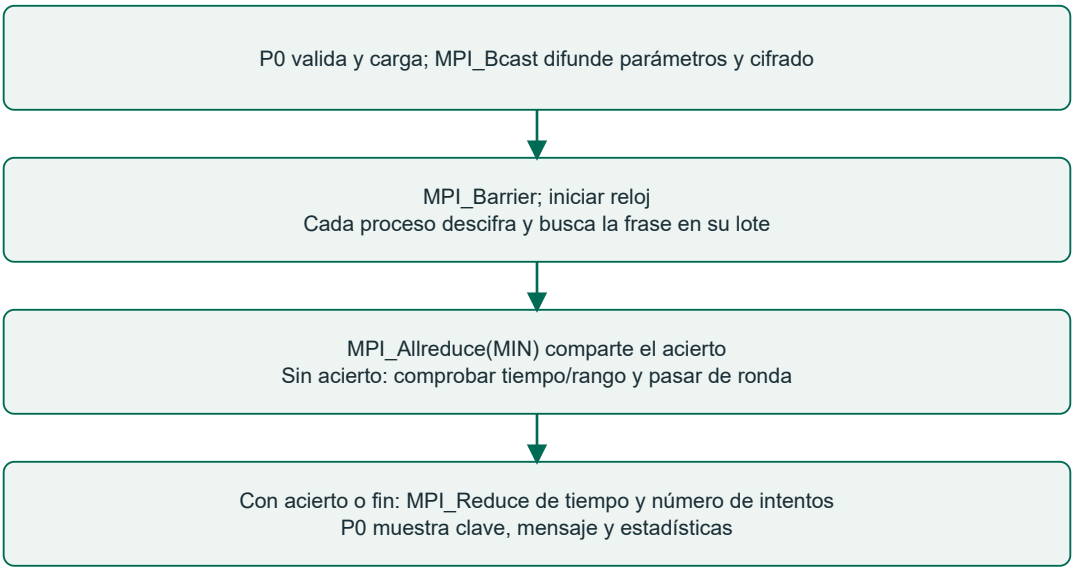


Figura 2. Flujo de la base. La repetición continúa mientras no haya acierto, límite o agotamiento.

Ventajas y limitaciones

El reparto es simple y no requiere un coordinador que asigne cada lote. Sin embargo, todos esperan la finalización de la ronda. Un proceso puede seguir trabajando mientras otro ya encontró la clave. Para 123456, el hallazgo ocurre en el lote de P1 de la primera ronda, pero los demás completan sus lotes antes del consenso.

6. Alternativa: familias de claves

La alternativa cambia el orden de exploración y la asignación de trabajo. Conserva la prueba de descifrado por candidata y añade cuatro familias, cada una con un contador de bloques. La familia se genera por índice; no se almacena una lista completa.

Familia	Candidatas y cobertura
F0: valores bajos	Enteros desde 0 hasta 2^24 - 1. Incluye 123456.
F1: patrones humanos	Fechas numéricas, dígitos repetidos, potencias de 2 y de 10 con sus vecinos y constantes. Incluye 2^54 y 2^54+1.
F2: texto ASCII	Cadenas de 1 a 5 caracteres de [a-z0-9] convertidas a un entero big endian.
F3: barrido de respaldo	Intervalo desde offset; por defecto offset = 2^24 y cantidad = 2^63. La cobertura depende de estos parámetros.

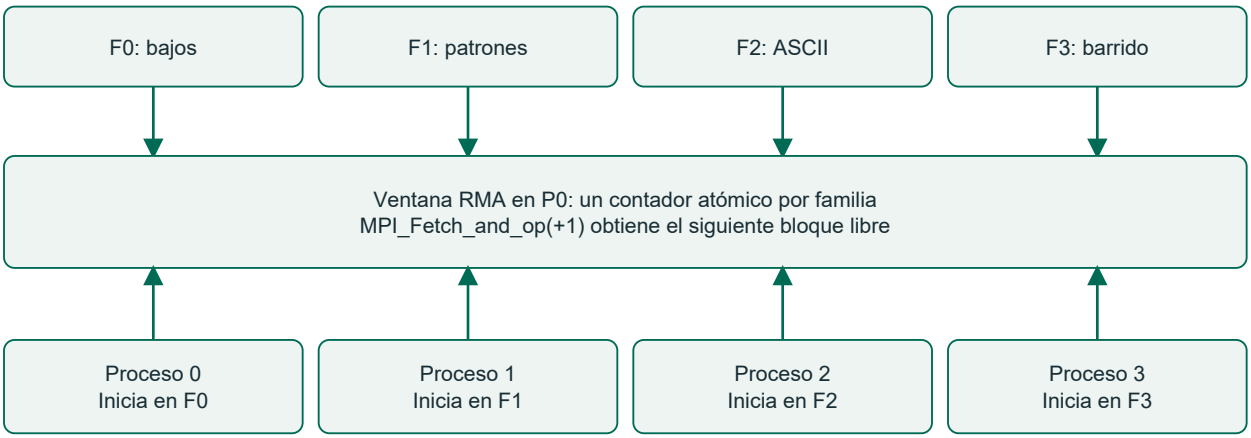


Figura 3. Asignación dinámica por familia. El tamaño habitual de bloque es 65536 candidatas.

Asignación y balanceo

P0 expone cuatro contadores mediante MPI_Win_create. Cada proceso incrementa el contador de su familia con MPI_Fetch_and_op y completa la operación con MPI_Win_flush. El valor anterior identifica un bloque exclusivo dentro de esa familia [5]. Cuando una familia se agota, el proceso solicita trabajo de las siguientes en orden circular.

La asignación evita duplicar índices de una misma familia. No elimina coincidencias entre familias: un entero puede aparecer como valor bajo, patrón o parte del barrido. El proceso conserva su preferencia de familia hasta agotarla; no hay redistribución continua basada en el tiempo restante de cada familia.

Modo de control -e

La opción -e desactiva F0, F1 y F2; todos los procesos solicitan bloques del mismo barrido. Con offset, límite y lote iguales a la base, permite comparar la asignación dinámica sin el efecto de priorizar patrones.

7. Comunicación y pseudocódigo

Antes de buscar, cada proceso deja una recepción MPI_Irecv pendiente. Si encuentra la frase y confirma el HMAC, envía la clave y familia con MPI_Issend [6]. Los receptores consultan MPI_Test cada 4096 intentos por familia y también entre bloques. El aviso reduce el trabajo posterior al acierto sin exigir una reducción por lote.

```
P0 valida y difunde parámetros y texto cifrado
crear ventana con contador[0..3] = 0
publicar recepción no bloqueante de aviso
sincronizar e iniciar reloj
mientras haya trabajo y no haya finalizado:
    elegir primera familia no agotada según preferencia del rank
    bloque = incremento atómico del contador de esa familia
    si bloque fuera de la familia: marcar agotada y continuar
    para cada índice del bloque:
        candidata = generar_clave(familia, índice)
        descifrar y buscar la frase
        si hay acierto:
            guardar clave y familia; enviar aviso a todos; finalizar
        cada 4096 intentos:
            consultar aviso y tiempo límite
    consultar aviso entre bloques
acordar clave mínima y familia mediante MPI_Allreduce
reducir tiempo máximo e intentos; drenar mensajes mientras MPI_Testall completa envíos
liberar ventana y contexto; P0 reporta; MPI_Finalize
```

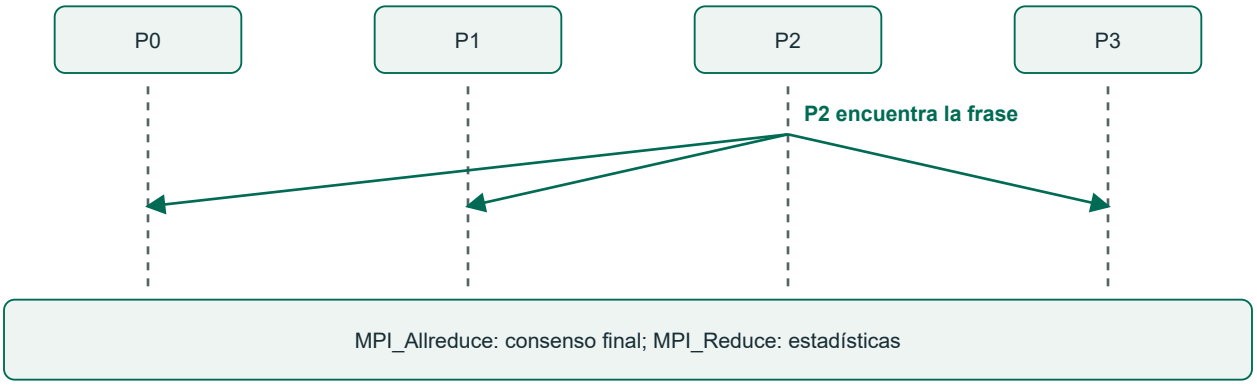


Figura 4. Ejemplo de aviso de P2 al resto. Los receptores detienen su búsqueda al detectar el mensaje.

Al terminar se cancela/completa la recepción pendiente y se drenan avisos mientras MPI_Testall progresa los envíos. La reducción final permite resolver aciertos simultáneos conservando la clave menor. El tiempo informado incluye el consenso, pero excluye la creación de la ventana y la limpieza posterior.

8. Metodología experimental

Variable	Configuración
Mensaje / frase	Esta es una prueba de proyecto 2 / es una prueba de. Se conservan los saltos del archivo.
Procesos y rango	4 procesos MPI. Base y secuencial: [0, 2^56), que incluye las tres claves.
Compilación	C11, -O2, -Wall, -Wextra, -Wpedantic; GCC 15.2.0; OpenSSL 3.5.5; Open MPI 5.0.10.
Sistema	Linux-6.18.33.2-microsoft-standard-WSL2-x86_64-with-glibc2.43
CPU visible a WSL	Intel(R) Core(TM) Ultra 5 125U
Serie principal	2026-10-27; 5 repeticiones por cada combinación. Corte de 5 s para las búsquedas largas.
Lote / sondeo	Principal: 65536 claves por lote; sondeo cada 4096 intentos. Control: lote 4096.
Fuentes	resultados.json, bitacora.txt y resultados.md. Metadatos de CPU, versiones, comandos y SHA-256 de fuentes.

Procedimiento y tiempo medido

Se cifra el texto con cada clave y se ejecutan las tres versiones cinco veces. Todos los códigos de salida y métricas se validan. Las búsquedas largas finalizan con código 3; las completadas con 0 y la clave esperada. La salida se guarda tras cada comando y un timeout externo invalida la medición. Se informa mediana, mínimo, máximo y desviación estándar muestral en resultados.md.

MPI_Wtime se inicia tras una barrera; la secuencial usa CLOCK_MONOTONIC. Se excluyen lectura, lanzamiento y reporte. El intervalo alternativo incluye el consenso de clave, pero excluye el drenaje y liberación posteriores. Son tiempos de búsqueda, no tiempos completos de la aplicación.

Control y límites de validez

Se agrega la clave 750123 con rango [0, 1000000), lote 4096 y alternativa en modo -e. Las tres versiones exploran el mismo espacio ascendente por bloques; la asignación y la parada pueden variar el trabajo efectivo. El entorno WSL es una máquina compartida: no hay aislamiento de carga ni estudio de varios nodos. Se registra la CPU visible, pero no se infiere de ella el hardware físico reservado.

9. Resultados medidos

Caso a): clave 123456

Programa	Mediana (s)	Rendimiento (claves/s)	Estado
secuencial	0.080493276	1,533,755	Hallada; n = 5
base	0.054260777	4,690,847	Hallada; n = 5
alternativa	0.096404394	4,628,486	Hallada; n = 5

La base obtiene una pequeña reducción del tiempo frente a la secuencial. La alternativa tarda más en este caso porque los otros procesos exploran familias que no contienen el acierto inmediato, mientras la búsqueda de valores bajos avanza hacia 123456. Un mayor total de claves por segundo no implica menor tiempo hasta el hallazgo si se prueban más candidatas.

Casos b) y c): alternativa

Caso	Clave	Mediana (s)	Familia / estado
b)	18014398509481984	0.071108893	Patrones; hallada, n = 5
c)	18014398509481985	0.063716091	Patrones; hallada, n = 5

Ambas claves se encuentran en la familia de potencias y vecinos. Son candidatas contiguas dentro de esa enumeración. La diferencia de sus medianas es pequeña y no permite concluir que aumentar en uno la clave produzca por sí mismo un cambio estable del rendimiento.

Muestras interrumpidas desde cero

Caso / programa	Tiempo real (s)	Intentos	Claves/s
b) secuencial	5.001534	9,900,032	1,979,846
b) base	5.020116	18,087,936	3,600,815
c) secuencial	5.000482	9,519,104	1,903,562
c) base	5.054247	17,563,648	3,466,635

Tabla 2. Medianas de cinco muestras por fila; código 3: tiempo agotado sin hallazgo. Cada campo se resume por separado.

Las capturas del Anexo 2 son sesiones independientes con cortes de 5 s. Sus tiempos individuales no sustituyen las medianas de esta serie.

10. Estimaciones y speedup

Proyección de las búsquedas exhaustivas

```
T_estimado(k) = (k + 1) / mediana(R_medido)
Supuestos: inicio en 0; rango [0, 2^56); tasa constante.
Años de referencia = segundos / (365.25 * 24 * 3600)
```

Caso	Programa	Proyección (s)	Años aprox.
b)	secuencial	9.09889e+09	288.3
b)	base	5.00287e+09	158.5
c)	secuencial	9.46352e+09	299.9
c)	base	5.19651e+09	164.7

Estos valores son extrapolaciones del rendimiento de una muestra corta a décadas o siglos de cómputo. No son tiempos medidos hasta el hallazgo, ni predicciones con incertidumbre cuantificada. En MPI la expresión aproxima el trabajo por rondas; para claves tan grandes, el lote final es despreciable frente al número proyectado de intentos.

Razones de tiempo

Caso	S base/sec	E base	G alt/base	Naturaleza
a)	1.48	0.371	0.563	Medida
b)	1.82	0.455	7.04e+10	Proyectada
c)	1.82	0.455	8.16e+10	Proyectada

Para a), S = 0.080493276 / 0.054260777 = **1.483** y E = **37.1 %**. La razón base/alternativa es **0.563**. Una razón menor que uno indica que la alternativa tarda más.

Las razones de b) y c) combinan un tiempo exhaustivo proyectado con uno heurístico medido. Expresan una ventaja condicionada por las familias y la extrapolación; no son un speedup paralelo puro por usar cuatro procesos.

10.1. Control de distribución

Programa	Mediana (s)	Mínimo (s)	Máximo (s)	Intentos med.
secuencial	0.460884	0.411778	0.552827	750,124
base	0.242585	0.216769	0.266417	750,124
alternativa	0.329376	0.283342	0.347526	758,316

Condiciones: clave 750123, mismo archivo, rango [0, 1000000), lote 4096, cinco repeticiones por versión, cuatro procesos en MPI y -e en la alternativa. Se desactivan las familias heurísticas para comparar el reparto sobre el mismo espacio. Los intentos pueden diferir por la asignación y la anticipación de la parada, que forman parte del comportamiento de cada estrategia.

La base obtiene S = **1.900** y eficiencia **47.5 %**. La razón base/alternativa es **0.736**; debe interpretarse con las muestras, los intentos y la dispersión conservada en resultados.md.

Este control permite separar la priorización de patrones de la asignación dinámica. No demuestra superioridad universal ni garantiza el mismo resultado con otro tamaño de lote, hardware o clave. La variación de WSL y las diferencias de sondeo pueden modificar tanto el número de intentos como el tiempo.

Correcciones verificadas

Área	Validación realizada
Archivos	Clave 44 rechazada; alteración de primer bloque, longitud y etiqueta rechazada sin salida.
Aritmética	Rangos que desbordan se rechazan; pruebas de búsqueda hasta UINT64_MAX - 1 sin vuelta a cero.
MPI	Aciertos simultáneos, tiempo límite común, solicitudes dimensionadas según procesos y drenaje de envíos.
Entorno	WSL/Open MPI 5: el lanzador trata retornos como normales y el helper recupera el consenso Estado MPI. Sin estado o ante timeout, falla.

11. Discusión de resultados

Influencia de la clave y el recorrido

En el recorrido ascendente, una clave baja se visita pronto y una clave alta requiere muchos intentos. El costo de descifrar con AES no crece por el valor decimal de la candidata; cambia su posición en la búsqueda. En la alternativa, una clave numéricamente grande puede visitarse pronto si tiene una estructura favorecida por las familias. La posición en la enumeración explica mejor el resultado que la magnitud aislada del entero.

Distribución y comunicación

La base hace trabajo extra hasta cerrar la ronda y el proceso con más trabajo determina la espera colectiva. La alternativa permite avisar antes y asignar bloques a quien los solicite, a cambio de operaciones RMA, sondeo y generación de patrones. Para búsquedas cortas esos costos pueden dominar. En a), el número de intentos de la base es mayor que el secuencial, por lo que el cociente de tasas no coincide con el speedup hasta encontrar la clave.

Ventajas y limitaciones de la alternativa

La priorización es útil para valores estructurados y evita fijar un offset junto a la clave en b) y c). Sin embargo, el conjunto de pruebas favorece precisamente las familias elegidas: una clave sin estructura puede depender del barrido de respaldo y tardar mucho más. Hay solapamiento de candidatas entre familias y la cobertura total depende de los límites; no se demuestra una aceleración general de AES-128.

Retos y soluciones implementadas

Se separó el código original del laboratorio, se definió un formato de archivo verificable, se compartió la validación MPI desde P0 y se reutilizó un contexto EVP por proceso. Para las ejecuciones impracticables se añadió un corte de tiempo y se conservaron rendimiento e intentos. Las capturas usan --mca osc ^ucx para evitar avisos de ese componente en WSL.

Correcciones y aprendizaje

Se corrigieron la aceptación de claves incorrectas con HMAC, la configuración del rango de medición, los desbordamientos en extremos de 64 bits, el consenso del tiempo agotado y el drenaje de aciertos simultáneos. Los parámetros se difunden con tipos MPI explícitos. En WSL se evita la ruta PRRTE de errores no-cero y el helper devuelve el consenso explícito Estado MPI del programa. La ausencia de ese estado invalida la ejecución; los timeouts no se reintentan. El aprendizaje es separar corrección, orden de búsqueda, coordinación y calidad de la evidencia antes de afirmar una mejora de rendimiento.

12. Conclusiones y recomendaciones

Conclusiones

1. Se completaron carga, cifrado y recuperación autenticada de archivos, referencia secuencial y dos estrategias MPI. La verificación incluye un vector AES-128-ECB, 24 casos de archivos y 75 casos de búsqueda, con entradas inválidas, extremos numéricos y aciertos simultáneos.

2. Para 123456, la base obtiene un speedup medido de 1.483 y eficiencia de 37.1 %. La razón base/alternativa es 0.563. Estos valores corresponden a medianas de cinco ejecuciones y no garantizan mejoras para toda clave.

3. Las medianas alternativas para 2^54 y 2^54+1 son 0.071109 s y 0.063716 s. La priorización de potencias y vecinos permite encontrarlas sin fijar el offset junto a la clave.

4. El control exhaustivo de la clave 750123 obtiene una razón base/alternativa de 0.736 con el mismo rango y lote. Es una comparación de distribución y parada sin priorización de familias.

5. Las búsquedas largas de base y secuencial están correctamente configuradas para incluir las claves, pero se interrumpen por tiempo. Las proyecciones no representan hallazgos medidos. Tampoco se recorrió el espacio completo de AES-128.

Recomendaciones y continuidad

1. Mantener los casos de regresión de autenticación, rangos y terminación en cada cambio; compilar y ejecutar las pruebas desde el ZIP antes de entregar.

2. Usar los JSON y comandos conservados para repetir las mediciones. Para estudios de rendimiento más precisos, aislar carga, registrar afinidad y repetir en máquinas o nodos dedicados.

3. Evaluar más claves sin estructura y tamaños de lote. Analizar dispersión e intentos junto con las medianas, sin atribuir al paralelismo la ganancia causada por el orden de candidatas.

4. Conservar AES-128-ECB como esquema didáctico del experimento. Para una aplicación real, emplear un diseño autenticado normalizado y claves aleatorias de entropía completa.

13. Referencias bibliográficas

[1] National Institute of Standards and Technology. (2023). Advanced Encryption Standard (AES), FIPS 197-upd1.

https://csrc.nist.gov/pubs/fips/197/final

[2] Dworkin, M. (2001). Recommendation for Block Cipher Modes of Operation: Methods and Techniques. NIST SP 800-38A.

https://csrc.nist.gov/pubs/sp/800/38/a/final

[3] OpenSSL Project. (s. f.). EVP_EncryptInit: EVP cipher routines, OpenSSL 3.5. Secciones de relleno y notas.

https://docs.openssl.org/3.5/man3/EVP_EncryptInit/

[4] Open MPI. (s. f.). MPI_Allreduce. Documentación de Open MPI 5.0.x.

https://docs.open-mpi.org/en/v5.0.x/man-openmpi/man3/MPI_Allreduce.3.html

[5] Open MPI. (s. f.). MPI_Fetch_and_op. Documentación de Open MPI 5.0.x.

https://docs.open-mpi.org/en/v5.0.x/man-openmpi/man3/MPI_Fetch_and_op.3.html

[6] Open MPI. (s. f.). MPI_Issend. Documentación de Open MPI 5.0.x.

https://docs.open-mpi.org/en/v5.0.x/man-openmpi/man3/MPI_Issend.3.html

[8] OpenSSL Project. (s. f.). HMAC. Documentación de OpenSSL 3.5.

https://docs.openssl.org/3.5/man3/HMAC/

[7] Universidad del Valle de Guatemala. (2026). CC3069 - Proyecto 02. Enunciado y rúbrica del curso Computación Paralela y Distribuida, Ciclo 2. Documento proporcionado con el proyecto.

Fuentes primarias del experimento

Repositorio del Proyecto 02: proyecto02/src/, proyecto02/include/ y proyecto02/tests/. Resultados: evidencias/inciso2/bitacora.txt, resultados.json y resultados.md. Capturas y transcripciones: evidencias/inciso1/ y evidencias/inciso2/. Registro final de pruebas: evidencias/pruebas/ultima_prueba.txt.

Las referencias web se consultaron para la elaboración del informe. Las explicaciones conceptuales están parafraseadas y señaladas con referencias numéricas en el cuerpo. Los diagramas y cálculos son de elaboración propia a partir del código y los registros.

Anexo 1. Catálogo de funciones

A1.1. AES y formato: proyecto02/include/aes_comun.h

Función	Entradas	Salida y funcionamiento
dataset_tag	Candidato, Dataset y etiqueta destino.	Deriva una clave MAC por dominio; calcula HMAC-SHA256 sobre versión, longitud y cifrado.
verify_dataset	Candidato y Dataset.	Recalcula y compara la etiqueta; 1 auténtico, 0 no válido, -1 error de biblioteca.
cipher_length	Longitud original.	Devuelve (longitud/16 + 1)·16; reserva relleno de 1 a 16 bytes.
make_key	Candidato uint64_t y arreglo de 16 bytes.	Llena el arreglo: ocho bytes iniciales en cero y entero en big endian.
parse_number	Cadena y puntero a entero.	1 si es decimal válido y distinto de UINT64_MAX; escribe el valor. 0 ante error.
encrypt_message	Contexto EVP, candidato, texto, longitud y Dataset.	Cifra con AES-128-ECB; llena Dataset, autentica y comprueba la longitud; devuelve éxito.
decrypt_message	Contexto, candidato, Dataset, búfer y puntero de longitud.	1 si HMAC, longitud y relleno son válidos; 0 si fallan; -1 ante error de inicialización/proceso. Escribe texto y longitud.
print_hex	FILE*, bytes y longitud.	Escribe hexadecimal minúsculo y salto de línea; no devuelve estado.
read_hex	FILE*, destino y cantidad de bytes.	Lee pares hexadecimales y exige salto final; 1 válido, 0 inválido.
save_dataset	Ruta y Dataset.	Escribe AESECB2, longitud, cifrado y etiqueta; devuelve éxito de E/S. Utilidad disponible, no usada por aes_archivo.
load_dataset	Ruta y Dataset de destino.	Valida cabecera, longitud, hexadecimal y fin de archivo; llena Dataset y devuelve éxito.

A1.2. Archivos: proyecto02/src/aes_archivo.c

Función	Entradas	Salida y funcionamiento
read_text	Ruta, destino y puntero de longitud.	Lee hasta 4096 bytes; rechaza excedentes o errores; conserva bytes y devuelve éxito.
write_output	Ruta, Dataset, texto y modo.	Abre con wbx; guarda contenedor o texto; elimina salida parcial si falla.
main	argc y argv: operación, entrada, salida, clave.	Coordina validación, EVP y E/S; limpia memoria. Código 0 si termina; 1 si falla.

Anexo 1. Búsqueda y MPI

A1.3. Comunes: proyecto02/include/busqueda.h y busqueda_mpi.h

Función	Entradas	Salida y funcionamiento
init_params	Params*, límite, lote y offset.	Inicializa valores, tiempo sin límite y frase por defecto.
parse_params	argc, argv, Params* y ruta de salida.	Lee -e, -f, -l, -b, -o y -t; rechaza rangos que desbordan y valida límites y un archivo; devuelve 1/0.
contains_phrase	Texto, longitud, frase y tamaño.	Compara ventanas con memcmp; devuelve 1 si contiene la frase.
try_phrase	EVP, candidato, Dataset, destino y frase.	Descifra, busca la frase y confirma con HMAC; 1 acierto, 0 ausente, -1 error EVP.
phrase_fits	Params y Dataset.	Devuelve si la longitud de la frase cabe en la longitud original declarada.
wall_seconds	Sin parámetros.	Tiempo double con CLOCK_MONOTONIC; termina con error si falla el reloj.
print_report	Modo, procesos, clave, intentos, tiempo, estado, EVP, datos y parámetros.	Muestra estado, clave hexadecimal, mensaje, tiempo y rendimiento.
mpi_setup	argc, argv, uso, Params y Dataset.	P0 valida y carga; difunde estado y datos mediante Bcast; todos reciben el mismo estado.
mpi_fail	Mensaje de error.	Escribe en stderr e invoca MPI_Abort para terminar el comunicador.
mpi_finish	Estado local de aplicación.	Consenso MAX, imprime Estado MPI en P0, finaliza y devuelve el estado. El helper WSL recupera ese código.

A1.4. Puntos de entrada de las búsquedas

Archivo / función	Entradas	Salida y funcionamiento
busqueda_secuencial.c / main	Opciones y archivo AES.	Recorre desde offset hasta el fin, prueba frase y mide tiempo. Libera EVP.
busqueda_base.c / main	Opciones y archivo AES; rank y tamaño MPI.	Reparte lotes, reduce el acierto por ronda y obtiene tiempo máximo e intentos totales.
busqueda_alternativa.c / main	Opciones y archivo AES; rank y tamaño MPI.	Crea contadores RMA, asigna bloques, sondea avisos, acuerda resultado y libera recursos.

Códigos de salida de las búsquedas: 0 = clave hallada; 1 = entrada o error; 2 = rango agotado sin hallar; 3 = tiempo agotado. Un aborto MPI puede producir un diagnóstico adicional del lanzador.

Anexo 1. Generadores y scripts

A1.5. Familias: proyecto02/src/busqueda_alternativa.c

Función	Entradas	Salida y funcionamiento
pow_u64	Base y exponente entero.	Producto iterado en uint64_t; calcula potencias usadas por los generadores.
key_patrones	Índice de la familia.	Devuelve fecha, repetición, potencia/vecino o constante según el segmento del índice.
ascii_total	Sin parámetros.	Suma 36^l para l de 1 a 5: cantidad de índices de la familia ASCII.
key_ascii	Índice ASCII.	Determina longitud, descompone en base 36 y empaqueta caracteres como entero.
family_count	Identificador de familia y Params.	Devuelve el número de índices: límite de bajos, patrones, ASCII o barrido.
family_key	Familia, índice y Params.	Selecciona el generador; en el barrido devuelve offset + índice.

A1.6. Automatización de pruebas y mediciones

Módulo / rutinas	Entradas y salidas; propósito
probar_inciso1.py: run, main	run ejecuta y comprueba éxito/error esperado; main crea archivos temporales, verifica bytes y 24 casos.
probar_busqueda.py: run, encrypt, found_key, main	run valida el código con timeout; encrypt genera un caso; found_key extrae la clave; main comprueba 75 casos; Open MPI es obligatorio.
ejecutar_pruebas.py: field, main; save y execute internas	field extrae métricas; execute valida estados; save conserva datos crudos; main mide serie principal y control con metadatos, JSON y tablas.
mpi_utils.py: mpi_prefix, run_command	Selecciona flags de Open MPI según entorno; ejecuta con timeout y termina el grupo completo si se excede.
registrar_pruebas.py: main	Ejecuta make test y guarda stdout, stderr, fecha y código en evidencias/pruebas/.
empaquetar.py: main	Reúne fuentes, ejecutables, informe y evidencias; genera ZIP y comprueba su integridad.

Alcance del catálogo: todas las funciones C de proyecto02 y las rutinas Python que verifican, miden y empaquetan la implementación. El Laboratorio 03 se conserva como antecedente; su API GCM no interviene en las búsquedas ECB del proyecto.

Anexo 1. Librerías y rutinas externas

Librería / API	Entradas, salidas y uso en el proyecto
OpenSSL: EVP_CIPHER_CTX_new / free	Crea un contexto o NULL; free lo libera. Un contexto independiente por proceso.
EVP_aes_128_ecb; EncryptInit/Update/Final	Selecciona cifrador; recibe clave y bytes; produce cifrado, longitudes y estado.
DecryptInit/Update/Final; set_padding	Recibe clave/cifrado y política de relleno; devuelve bytes y estado. Final verifica el relleno si está habilitado.
OPENSSL_cleanse	Puntero y cantidad de bytes; limpia búferes antes de liberar o descartar datos.
MPI_Init / Finalize; Comm_rank / size	Inicializa/finaliza el entorno y consulta identificador y cantidad de procesos.
MPI_Bcast / Barrier / Wtime	Difunde estado y datos desde P0; sincroniza el inicio; devuelve reloj local en segundos.
MPI_Allreduce / Reduce	Recibe valores, tipo y operador; combina MIN de claves, MAX de tiempo/estado y SUM de intentos. Allreduce devuelve a todos; Reduce a P0.
MPI_Win_create / lock_all / unlock_all / free	Expone contadores, abre/cierra época de acceso y libera ventana. Devuelve manejador/estado.
MPI_Fetch_and_op / Win_flush	Recibe incremento, destino y desplazamiento; retorna contador anterior y asegura finalización remota.
MPI_Irecv / Issend / Test	Publica recepción/envíos de clave y familia; devuelve solicitudes. Test consulta finalización y datos recibidos.
MPI_Testall / Cancel / Wait	Progresa y consulta envíos mientras se drenan avisos; cancela/completa la recepción pendiente.
HMAC / EVP_sha256 / CRYPTO_memcmp	Derivación por dominio y etiqueta SHA-256 de 32 bytes; comparación de autenticación sin memcmp ordinario.
MPI_Iprobe / Recv / Abort	Consulta y drena mensajes adicionales; Abort termina el comunicador ante error no recuperable.
C: stdio, stdlib, string, stdint, inttypes, errno	Archivos y E/S, conversión decimal, memoria/cadenas, enteros y formato portable, detección de errores.
POSIX: getopt y clock_gettime	Procesa opciones desde argv y obtiene reloj monotónico. Encabezados unistd.h y time.h.
Python: pathlib, subprocess, tempfile, re, statistics, zipfile	Rutas, procesos, archivos temporales, extracción de métricas, mediana y empaquetado. ReportLab genera este informe.

Dataset conserva longitud, cifrado y etiqueta HMAC. Params contiene rangos, tiempo, modo y frase. La difusión usa MPI_UINT64_T, MPI_INT y caracteres explícitos; no transmite el padding de estructuras nativas.

Anexo 2. Evidencia de compilación

Las capturas reales de xterm se obtuvieron con Xvfb, en un namespace privado para no alterar WSLg. Cada una tiene transcripción .txt y hashes en evidencias/capturas.json. Son sesiones independientes de la serie de medianas.

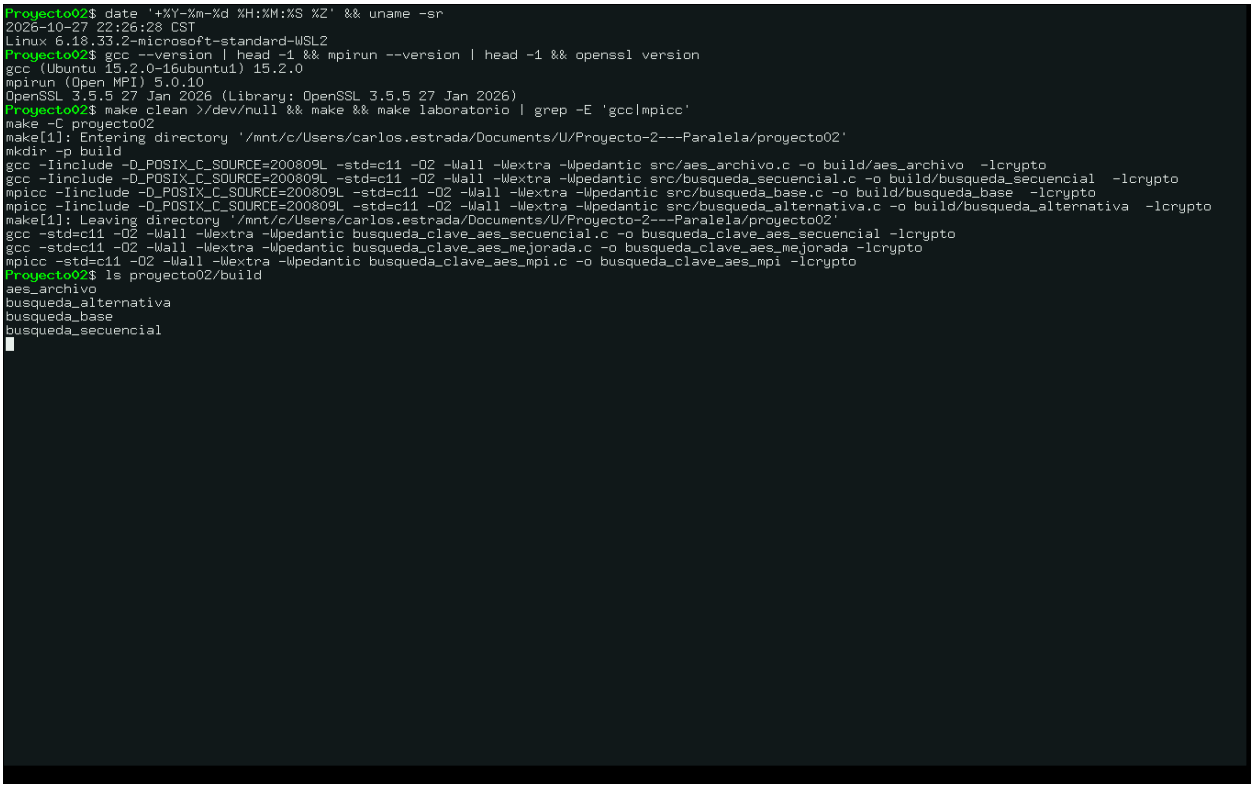


Figura A2.1. Entorno WSL, compilación de los cuatro ejecutables del proyecto y programas originales. Fuente: evidencias/inciso1/01_compilacion.png y compilacion.txt.

Interpretación

Se observan los comandos make clean, make y make laboratorio, así como GCC y mpicc con enlace a libcrypto. Se generan aes_archivo, busqueda_secuencial, busqueda_base y busqueda_alternativa. La captura contiene la versión GCC 15.2.0, Open MPI 5.0.10 y OpenSSL 3.5.5.

Anexo 2. Evidencia de archivos



Figura A2.2. Texto original, clave 123456 y representación hexadecimal del cifrado. Fuente: evidencias/inciso1/02_cifrado.png y cifrado.txt.

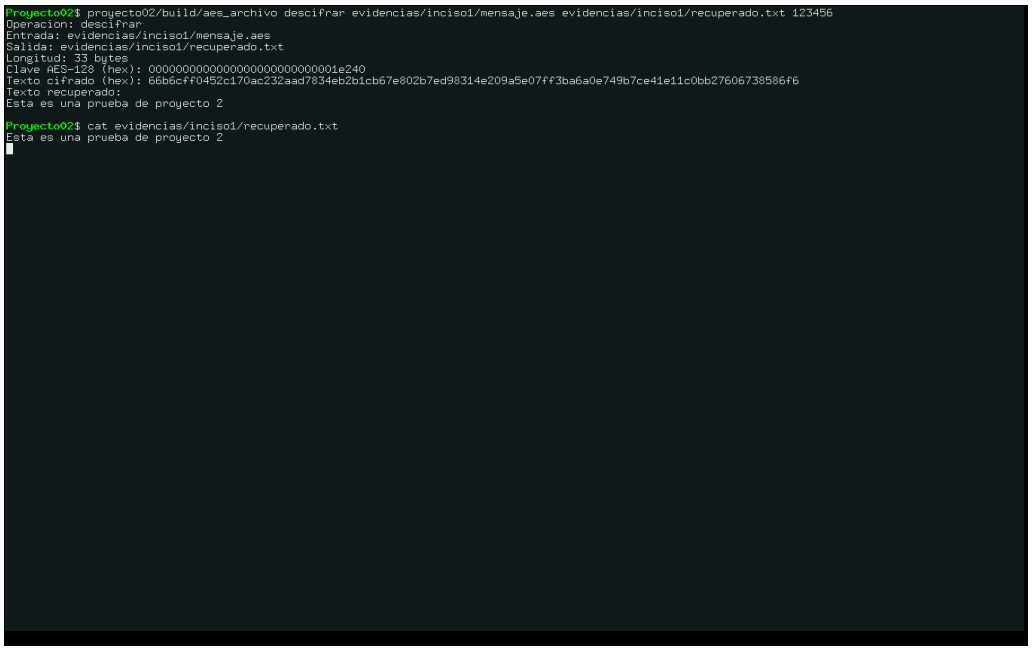


Figura A2.3. Descifrado con la misma clave y lectura del archivo recuperado. Fuente: evidencias/inciso1/03_descifrado.png y descifrado.txt.

Estas ejecuciones satisfacen la demostración de entrada, cifrado y recuperación del inciso 1. La equivalencia byte a byte se comprueba en la siguiente página.

Anexo 2. Verificación y pruebas

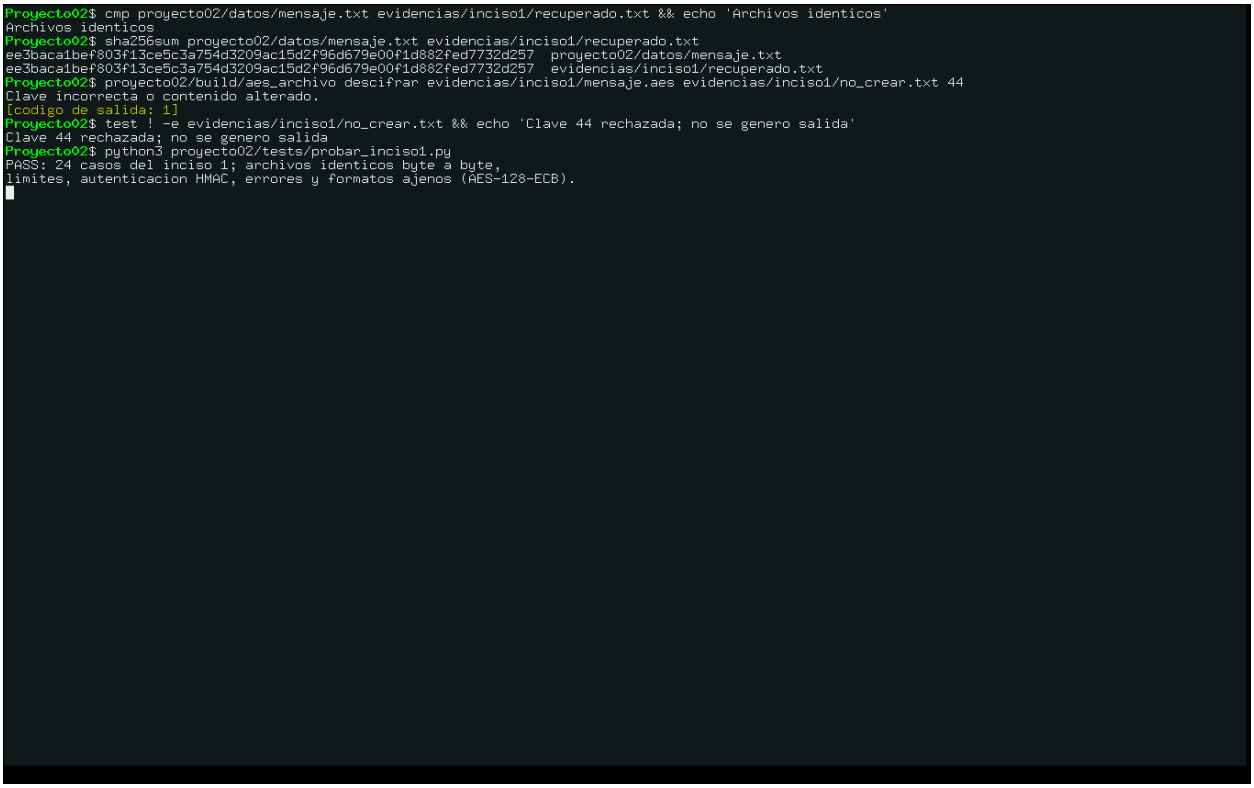


Figura A2.4. cmp, hashes SHA-256 iguales, rechazo de la clave incorrecta 44 y 24 casos de archivos. Fuente: evidencias/inciso1/04_verificacion.png y verificacion.txt.

Registro final de pruebas

```
Fecha: 2026-10-27T22:28:20-06:00
PASS: vector AES-128-GCM y rechazo de etiqueta alterada.
PASS: vector AES-128-ECB, recuperacion y autenticacion de clave/contenido.
PASS: 24 casos del inciso 1; archivos identicos byte a byte,
limites, autenticacion HMAC, errores y formatos ajenos (AES-128-ECB).
Pruebas de busqueda (incisos 2 y 3): 75 casos correctos
Código de salida: 0
```

Extracto de evidencias/pruebas/ultima_prueba.txt. El vector GCM corresponde al laboratorio y el vector ECB verifica el cifrado utilizado por el proyecto.

Las regresiones cubren autenticación de contenido, longitud y etiqueta, límites de archivos y rangos, frases cortas, tiempo límite y aciertos simultáneos. El helper MPI conserva los códigos reales: 1 para entrada inválida, 2 para ausencia y 3 para tiempo agotado. Un timeout externo provoca un fallo y no se contabiliza como prueba superada.

Anexo 2. Caso a): clave 123456

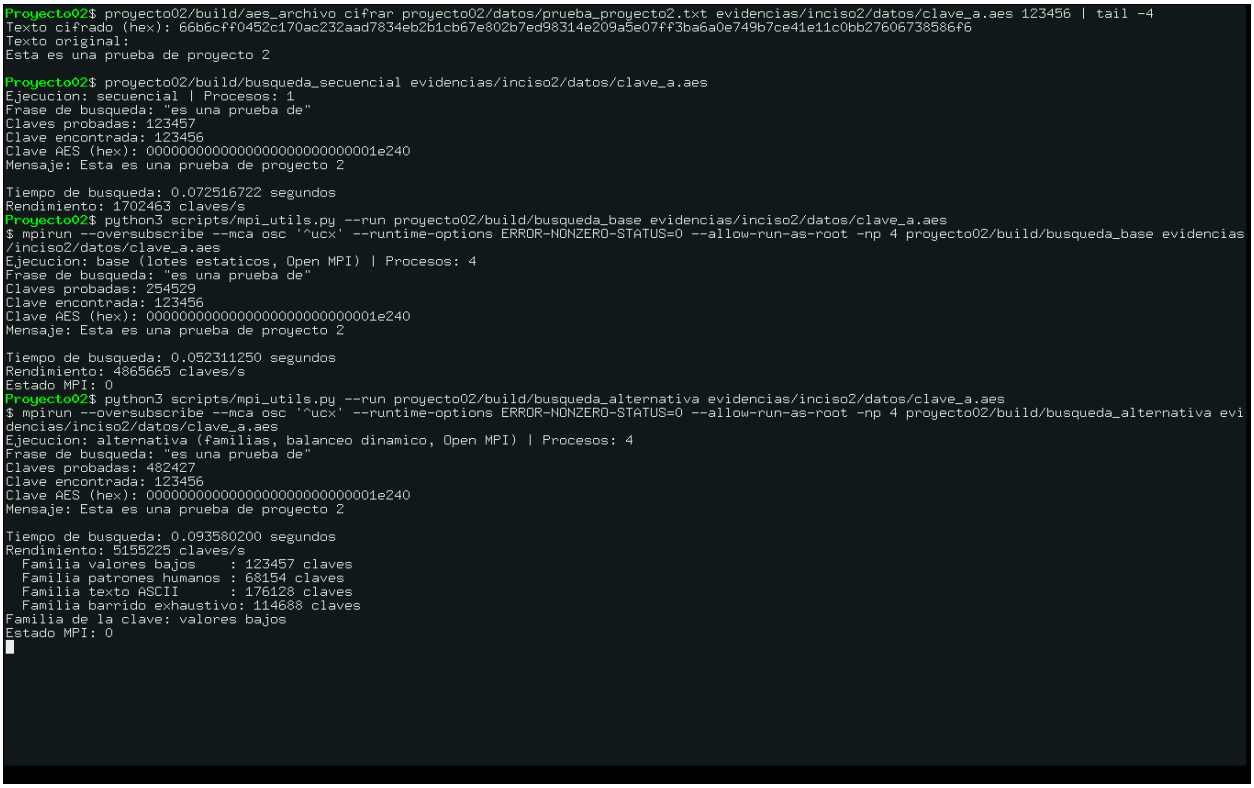


Figura A2.5. Ejecución de las tres versiones para la clave 123456. Fuente: evidencias/inciso2/01_clave_a.png y clave_a.txt.

Las tres versiones encuentran 123456. La captura corresponde a una ejecución individual de evidencia y no sustituye la mediana de las cinco muestras de la serie principal. Los comandos, la clave y la frase son visibles.

Anexo 2. Caso b): clave 18014398509481984

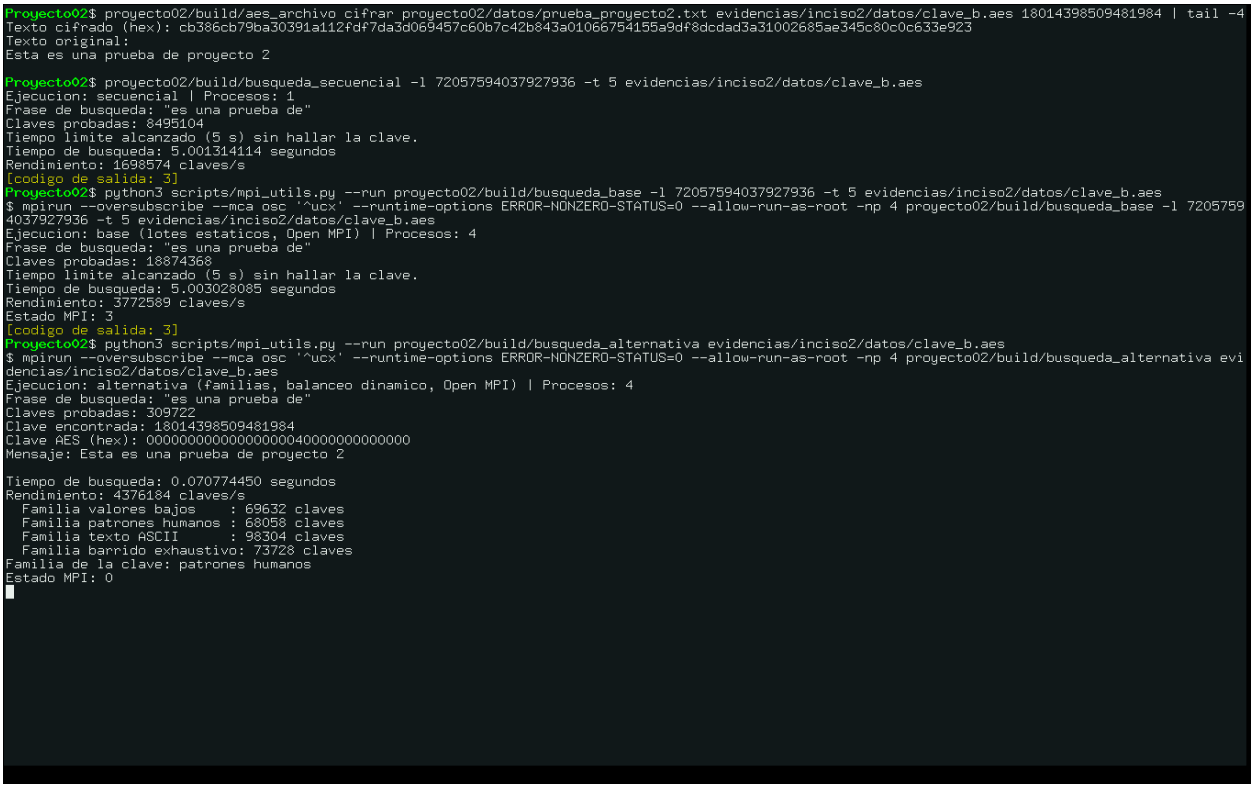


Figura A2.6. Ejecución de las tres versiones para la clave 18014398509481984. Fuente: evidencias/inciso2/02_clave_b.png y clave_b.txt.

La secuencial y la base se interrumpen tras aproximadamente 5 s sin hallar la clave y devuelven 3. El rango explícito [0, 2^56) incluye la candidata objetivo. La alternativa encuentra la clave en la familia de patrones sin conocerla como argumento de búsqueda. La configuración de PRRTE evita su ruta problemática de errores no-cero; el helper recupera el código real desde Estado MPI, mostrado en la salida.

Anexo 2. Caso c): clave 18014398509481985

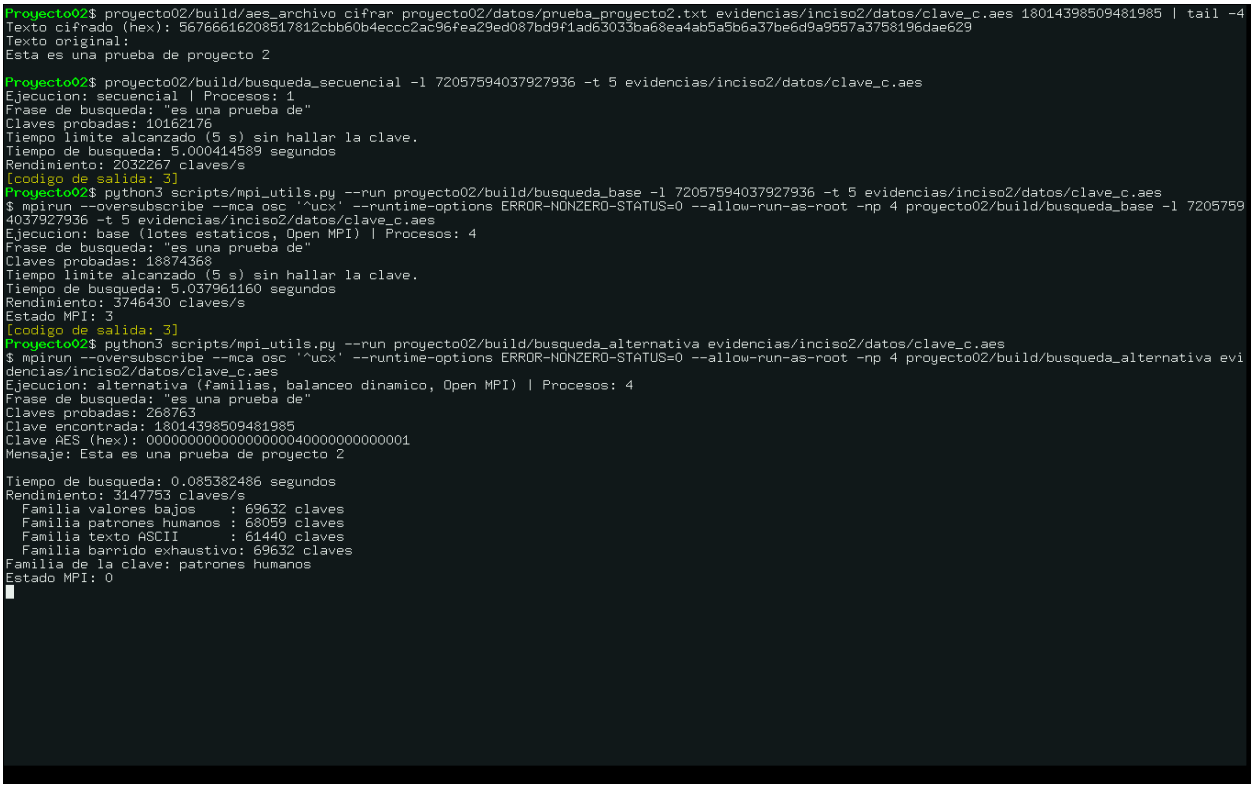


Figura A2.7. Ejecución de las tres versiones para la clave 18014398509481985. Fuente: evidencias/inciso2/03_clave_c.png y clave_c.txt.

La secuencial y la base se interrumpen tras aproximadamente 5 s sin hallar la clave y devuelven 3. El rango explícito [0, 2^56) incluye la candidata objetivo. La alternativa encuentra la clave en la familia de patrones sin conocerla como argumento de búsqueda. La configuración de PRRTE evita su ruta problemática de errores no-cero; el helper recupera el código real desde Estado MPI, mostrado en la salida.

Anexo 2. Bitácora de mediciones

Serie principal: 2026-10-27. Las columnas enumeran las cinco ejecuciones de la bitácora y la mediana calculada sin mezclar las sesiones de capturas.

Caso / programa	1 (s)	2 (s)	3 (s)	4 (s)	5 (s)	Mediana
a) secuencial	0.062624	0.080493	0.073371	0.086253	0.088182	0.080493
a) base	0.054261	0.054739	0.055287	0.050402	0.052852	0.054261
a) alternativa	0.101446	0.091841	0.096404	0.085552	0.122602	0.096404
b) alternativa	0.072209	0.076513	0.071109	0.065999	0.056482	0.071109
c) alternativa	0.076677	0.083911	0.053489	0.058530	0.063716	0.063716

Trazabilidad de las ejecuciones largas

Caso / programa	Tiempo (s)	Intentos	Salida
b) secuencial	5.001533946	9,900,032	3: tiempo agotado
b) base	5.020116443	18,087,936	3: tiempo agotado
c) secuencial	5.000481566	9,519,104	3: tiempo agotado
c) base	5.054247210	17,563,648	3: tiempo agotado

Cada fila larga resume 5 ejecuciones con corte de 5 s y rango [0, 2^56). Se conservan las muestras individuales, tasas, intentos y estados en resultados.json. Las medianas de intentos y de tiempo se calculan por separado, por lo que su cociente no necesariamente es la mediana de tasas.

Cálculo reproducible de un ejemplo

```
Caso b), base: R = 3600815 claves/s
T_proyectado = (18014398509481984 + 1) / 3600815
T_proyectado = 5.00287e+09 s
Condición: rango ampliado y rendimiento constante.
La ejecución registrada no encontró la clave.
```

Archivos completos de soporte

bitacora.txt conserva comandos, salidas, intentos, tasas y códigos de salida; resultados.md conserva la tabla original; datos/clave_a.aes, clave_b.aes y clave_c.aes permiten comprobar cada entrada. Este informe mantiene las cifras originales y explicita sus condiciones de interpretación.

Apéndice. Reproducción y entrega

Desde una extracción nueva en Ubuntu o WSL

```
sudo apt install build-essential libssl-dev libopenmpi-dev openmpi-bin python3
make
make test
python3 proyecto02/tests/ejecutar_pruebas.py 5 5
python3 scripts/registrar_pruebas.py
python3 informe/crear_informe.py
python3 scripts/empaquetar.py
```

Para generar el informe se necesitan ReportLab, Pillow y fuentes Arial o DejaVu Sans/Mono. Las pruebas y mediciones no requieren las librerías de PDF. Los archivos de salida AES deben ser nuevos. Para las claves grandes, los comandos de cifrado reciben el entero; la búsqueda solo recibe el archivo, la frase y los límites del recorrido, nunca la clave objetivo.

Demostración de búsqueda con cuatro procesos

```
python3 scripts/mpi_utils.py --run proyecto02/build/busqueda_base \
  -l 72057594037927936 -t 5 evidencias/inciso2/datos/clave_b.aes
python3 scripts/mpi_utils.py --run \
  proyecto02/build/busqueda_alternativa \
  evidencias/inciso2/datos/clave_b.aes
```

Estado final y límites del experimento

Área	Estado
Programa	Cifrado autenticado, frase con confirmación, rangos sin desbordamiento y terminación MPI verificados.
Pruebas	Vector AES-128-ECB, 24 casos de archivos y 75 casos de búsqueda; make test exige Open MPI.
Mediciones	Cinco muestras por combinación, rango correcto, metadatos y control sin heurísticas. Proyecciones separadas.
Entrega	Fuentes, ejecutables Linux, informe, evidencias, manifiesto SHA-256 y registro final de pruebas.
Límites	Las búsquedas exhaustivas grandes siguen interrumpidas por costo. No se afirma haberlas completado ni haber roto AES-128.

Las ampliaciones propuestas en recomendaciones corresponden a futuros estudios de rendimiento. Los fallos funcionales identificados durante la revisión fueron corregidos y se incorporaron regresiones.